

Android Permissions: Evolution, Attacks, and Best Practices

Güliz Seray Tuncay  | Google

In this article, we study the evolution of Android permissions. We describe the rationale behind key changes in Android's permission model and disclose two permission-related security vulnerabilities we discovered. Finally, we provide developers actionable insights to proactively address permission-related security and privacy risks during development.

Android permissions are a fundamental real-world security protocol that is currently deployed on billions of Android-powered devices, ranging from smartphones and tablets to smartwatches and automobiles. Android's permission model constitutes one of the cornerstones of security and privacy on the Android platform and is a security protocol that Android users are familiar with due to their regular interactions with applications (apps).

More specifically, Android utilizes its permission model to regulate access to sensitive user data (e.g., contacts, location), device resources (e.g., camera, microphone), and critical actions that can be performed on the platform (e.g., installing new apps, overlaying on top of other apps). Permissions are categorized based on the risk associated with the resource or the action the permission is guarding, and might be granted by the platform in different ways. While some permissions might be granted automatically at installation, others require user consent to be obtained before an app can access the respective resource. There are also permissions that are reserved solely for first-party use (i.e., the platform, first-party, or preinstalled apps) and thus they cannot be granted to third-party apps.

Throughout the years, there have been numerous updates to Android's permission model, ranging from isolated changes that might require only a subset of apps to make minor modifications to adapt, to substantial structural changes that significantly affect how apps and users deal with permissions. For example, the `READ_LOGS` permission was restricted for use by third-party apps in Android 4.1 due to privacy concerns. This change does not require any major structural code modifications by most app developers and does not affect how users interact with the system or their apps in general. In contrast, Android 6.0 introduced the runtime permission model, which brought a paradigm shift in how user consent is obtained (i.e., at runtime instead of during installation) and significantly challenged app developers to restructure their apps to conform to the new model. Android 10 and 11 further restricted access to sensitive resources based on the app's context (e.g., visibility). With this change, permissions became contextual with a non-binary state, allowing users to grant apps access to resources under specific conditions (e.g., only while using the app or for a single use). As of now, these context-based restrictions apply only to location, microphone, camera, and body sensors.

In this article, we study Android's permission model and its evolution over time. In particular, we explain

Digital Object Identifier 10.1109/MSEC.2024.3461629

key changes made by Google to Android permissions, the rationale behind these changes, and their impact on users and app developers. We then describe two serious security vulnerabilities we discovered in our work. Finally, we offer guidelines for platform developers to help them avoid inadvertently introducing security issues during the design and implementation of permission changes. We also offer recommendations to app developers to help them securely handle sensitive data in their apps.

Permission Fundamentals

Android is a mobile operating system primarily developed and maintained by Google and its source code is made available via the Android Open Source Project (AOSP).^a Android permissions constitute one of the cornerstones of security and privacy on the Android platform and have been subjected to numerous changes since the first commercial introduction of Android in 2008. This section covers some of the fundamental permission-related concepts.

Application Sandbox

Android is built on top of the Linux kernel and utilizes Linux concepts to establish isolation between applications. In particular, Android adapts Linux's user isolation mechanism, originally intended for separating physical users, to create a secure sandbox for Android applications. Each app is assigned a unique and permanent user ID (UID) and executed as a process running within its application sandbox. A sandboxed app is given a private data directory with read and write access. To access resources outside its sandbox, such as hardware devices, user data, and the Internet, an Android app requires specific permissions.

Permission Structure

Permission types. There are two kinds of permissions on Android: *system* and *custom* permissions. System permissions are declared within the platform and published in the platform application programming interface (API) reference. They guard sensitive actions (e.g., installing a new app), user data (e.g., contacts), and device resources (e.g., camera). Custom permissions, on the other hand, are declared by app developers to enable sharing and collaboration among apps. Using custom permissions, app developers can give other apps controlled access to certain components within their apps.

Permissions can be defined either statically or dynamically on Android. Listing 1 illustrates the static definition of the READ_SMS system permission. Listing 2 illustrates the static definition of a custom permission

```
<!--Allows an application to read SMS messages.-->
<permission
    android:name="android.permission.READ_SMS"
    android:permissionGroup=
        "android.permission-group.MESSAGES"
    android:protectionLevel="dangerous"
    android:label="@string/permlab_readSms"
    android:description="@string/permdesc_readSms"/>
```

Listing 1. Permission definition inside the Android Platform

```
<!--Define custom permission.-->
<permission
    android:name="com.test.MY_OWN_PERMISSION"
    android:protectionLevel="signature"
    android:permissionGroup="com.test.
        MY_OWN_PERMISSION_GROUP"/>

<!--Protect component with custom permission.-->
<service
    android:permission="com.test.MY_OWN_PERMISSION"
    android:name=".MyService"/>
```

Listing 2. Custom permission in an app's AndroidManifest.xml

and its assignment to an app component inside an app's manifest file. In this example, the custom permission is used to restrict access solely to apps sharing the same developer, verified by having the same signing certificate. Android additionally allows apps to dynamically create custom permissions via the `addPermission()` API. For this, apps must declare permission trees in their manifest file, specifying the domain name under which the dynamic permissions will be created.

Protection levels. Each permission is associated with a protection level that indicates the level of risk incurred in granting the permission to apps. The protection level also affects the procedure followed by the platform to grant or deny the permission. Here are Android's protection levels:

- **normal:** This protection level indicates low risk in granting the permission to apps. Some actions considered to be low risk on Android are accessing the Internet, setting an alarm, and connecting to paired Bluetooth devices. An app requesting a permission with this protection level is automatically granted the permission at installation.
- **dangerous:** This protection level is assigned to permissions that protect high-risk resources, such as camera, microphone, location, etc. The platform grants this permission only after obtaining user consent, which occurs at installation before Android 6.0 and at runtime afterwards.
- **signature:** The platform grants a permission with this protection level only if the app requesting the

^a<https://source.android.com>

permission is signed with the same certificate as that of the app that declared the permission. If the certificates match, the platform automatically grants the permission at installation to the requesting app without requiring user consent.

- **signatureOrSystem:** A permission with this protection level is granted either to an app signed with the same certificate as that of the app declaring the permission or to an app that resides in a designated folder on the system image. This protection level was intended to handle the special situation where preinstalled apps coming from multiple vendors are signed by different certificates but need to share features since they are built together. It is deprecated in Android 6.0 and the system developers are advised to use `signature|privileged` instead.
- **internal:** This protection level was added in Android 12. Unlike the previous protection levels, permissions with this protection level are never granted automatically. Instead, the system grants them based on the permission's protection level flags. This protection level is commonly used for managing permissions that are coupled with roles,^b which are unique positions within the system that are associated with a designated functionality (e.g., assistant, browser) and can hold certain permissions and privileges.

The rest of this article will follow the convention used by Android platform developers and refer to permissions with their protection levels for brevity (e.g., normal permissions). Note that the current Android documentation generally refers to permissions of dangerous level as *runtime permissions* instead of *dangerous permissions*, as these are synonymous starting from Android 6.0. However, in this article, we will be using the latter for consistency.

Protection level flags. The protection level can be supplemented with protection level flags to indicate additional behavior. Protection level flags are assigned to permissions in the form of `level|flag` in the protection level field of the permission definition. Among the existing protection level flags, a crucial one, particularly from the user's perspective, is the `appop` flag. This flag safeguards data considered even more sensitive than those protected by dangerous permissions, necessitating more positive friction to be granted to help users better manage their privacy risk. For example, displaying over other apps, installing unknown apps, and all file access are some of the actions that are currently protected under the `appop` (also known as *special access*) permissions. The platform requires the user to visit the settings to grant these permissions to apps. Another

important flag is `privileged`, which indicates the permission will be granted to apps that are installed as privileged apps on the system image. Additionally, the `role` flag signals that the permission will be managed based on roles. Android developer guides describe the behavior of Android's protection level flags.¹

Permission flags. Permission flags were first introduced in Android 4.2 to provide more context for a permission. For example, `costsMoney` was introduced to indicate that the operation might cost users money, such as when sending SMS to premium numbers. Android developer guides show the list of permission flags available on Android.²

Permission groups. Android groups permissions based on the functionality the permission is associated with. For example, permissions that are associated with text messaging (e.g., `SEND_SMS`, `RECEIVE_SMS`, `RECEIVE_WAP_PUSH`, `RECEIVE_MMS`) are all grouped under the SMS permission group. Similarly, permissions associated with contacts (e.g., `READ_CONTACTS`, `WRITE_CONTACTS`) are grouped under the CONTACTS permission group. Android developer guides show the list of permission groups available on Android.³

Requesting and Enforcing Permissions

Requesting permissions. Apps needing access to resources guarded by permissions should declare their required permissions in their manifest. Before Android 6.0, all permissions requested by an app were automatically granted if the user proceeded with the installation. Starting with Android 6.0, apps request permissions at runtime, where users are prompted with permission dialogs and asked to give consent to sensitive resource access by apps. Users are also given the ability to adjust app permissions through the Settings at any time. We will further elaborate on this change in the next section.

Enforcement. Permissions are enforced at different levels of the operating system. Access to low-level components, such as local sockets and device files, is regulated within the Linux kernel based on the UID or group ID of the requester. Higher-level resources, such as location and microphone, are enforced at the framework layer within the Android system services.

Evolution of Permissions

Android's permission model continuously evolves to address the dynamic needs of the mobile landscape and ensure robust security and privacy for its users. Platform developers have implemented various strategies, focusing on empowering users with greater control over their data, improving the usability of existing privacy controls, and

^b<https://source.android.com/docs/core/permissions/android-roles>

refining access control mechanisms to achieve the principle of least privilege. These concerted efforts have led to significant changes in Android's permission system, which we discuss in more detail in this section.

Changes in Permission Paradigms

Runtime permissions. Android 6.0 introduced significant changes to how permissions are handled on the platform. Previously, all permissions, regardless of their protection levels, were granted to apps at installation. In this scheme, users were given a chance to review an app's required permissions on the app market before deciding to install it, as shown in Figure 1(a). With Android 6.0's runtime permission model, dangerous permissions are no longer granted at installation. Instead, they will be requested by apps at runtime, where the user will be shown a designated permission user interface and asked to decide whether or not to grant the permission to the requesting app, as shown in Figure 1(b).

This change happened in an effort to clearly separate security and privacy for Android users. Regarding security, users mainly care about whether they are safe or not (e.g., if they have installed any harmful apps), while their mental model for privacy is complex as it revolves around being able to adjust what each app can access.⁴ Additionally, with the install-time model, users who did not agree with all of the permissions an app requested were left with no choice but to not install the app, or uninstall it if they changed their minds later because of the app's permissions. The negative impact of this restriction on users was noticeable, especially for apps without alternatives, such as social media apps. In addition, it was becoming infeasible for users to review all of the permissions needed by an app at installation given that the number of permissions in the system was steadily increasing, as will be shown later in this section.

All of these issues necessitated the introduction of runtime permissions, with the aim to simplify how users dealt with permissions and reduce warning fatigue. This was achieved first by identifying the critical resources users care about and limiting the obtainment of user consent to these resources. Then, the permissions for these resources were grouped based on their functionality and relevance to each other and were requested dynamically on a group basis. In other words, users will be prompted for a permission group instead of each individual permission in that group and an app will be automatically granted all of its requested permissions that belong to the same permission group when the user grants one of them.

Multi-state permissions. Android 9.0 was the first version to limit apps' access to sensors, microphone, and camera in the background. However, this change did

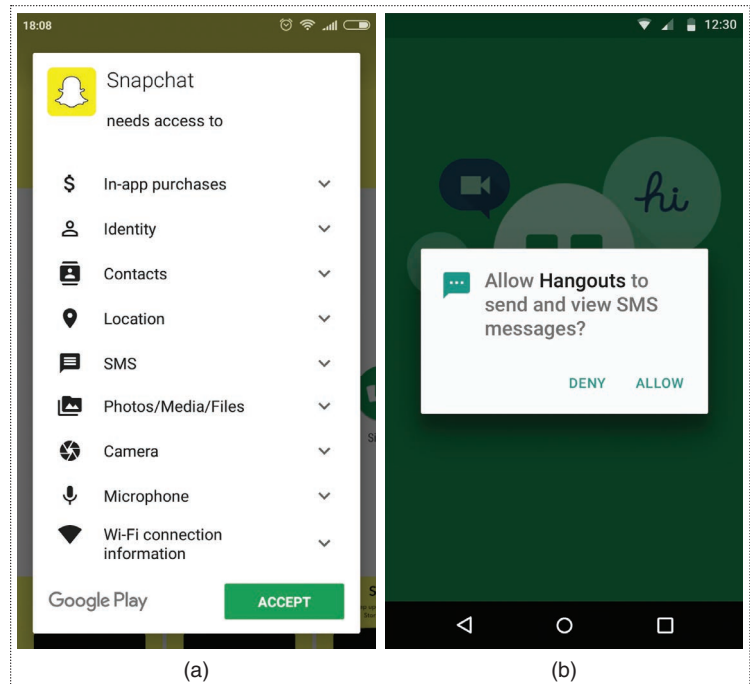


Figure 1. Android's evolving permission model. Previously, all permissions were granted at installation (a). Android 6.0 introduced runtime permissions (b), which required explicit user consent at runtime for highly-sensitive resources.

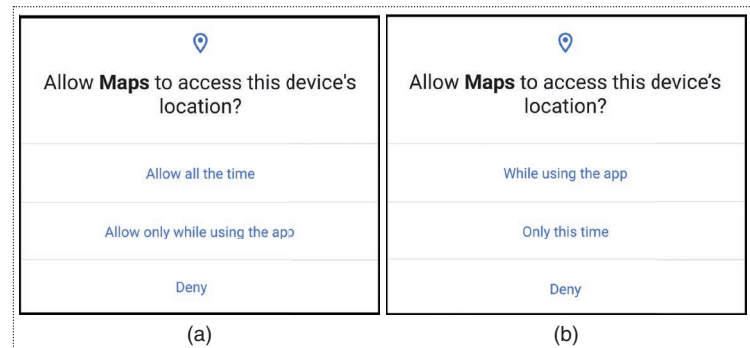


Figure 2. Multi-state permissions on Android. Android 10 adopted while-in-use permissions (a), while Android 11 additionally introduced one-time permissions (b).

not affect how users interacted with permissions and was primarily introduced to improve power efficiency by limiting the frequency of background data access. Android 10 introduced restrictions on background location access as a privacy enhancement, modifying the location permission dialog to allow users to choose when an app can access their location.^c Specifically, users were given the option to grant an app only *foreground access*^d (i.e., the app has an Activity in the foreground or it has an active foreground service) to

^c<https://developer.android.com/about/versions/10/privacy/changes>

^d<https://developer.android.com/develop/sensors-and-location/location/permissions>

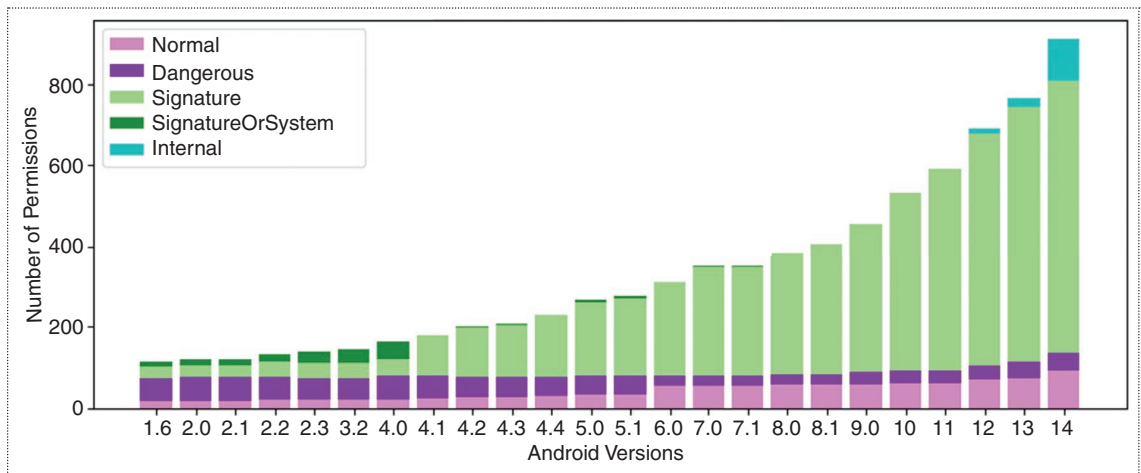


Figure 3. Number of permissions and their protection levels across Android versions. The number of permissions increase with each Android version to respond to the changing demands of the platform and the privacy expectations of the users. Note that the permissions that use `signature|system` or `signature|privileged` in the later versions instead of the deprecated `signatureOrSystem` are classified as `signature`.

their location, as shown in Figure 2(a). This concept, which we refer to as *while-in-use permissions*, was later extended to camera, microphone, and body sensors. With this change, permissions became multi-state on Android.^e

Android 11 further extended the restrictions on location, microphone, and camera access, allowing users to grant apps permission for a “single use”; that is, the permissions for these resources will be revoked once the app is no longer in the foreground. This feature is commonly known as *one-time permissions*. Figure 2(b) depicts the respective permission dialog for location in Android 11. As can be seen from this figure, the option “allow all the time” is removed from the permission dialog in this version. To grant background access to apps, users need to manually grant it through the settings. This change was implemented because the “while using the app” option was found to cover the vast majority of app use cases, owing to its inclusion of foreground services.

It is worth noting that the concepts of *foreground* and *background* are never directly exposed to users, as these can be ambiguous from the user’s perspective. Instead, Android uses a simpler language to more clearly describe the context of a permission grant.

Permission Statistics

Number of permissions. Figure 3 shows the change in the number of permissions defined in the platform and their protection levels across Android versions.^f

As can be seen, the number of permissions consistently increases with each Android release. Most of this increase has been due to the introduction of new signature and internal permissions because of efforts to compartmentalize system components to achieve least privilege and do not directly, or at least significantly, affect users or third-party app developers.⁵ Besides, we observe a consistent trend of increase, albeit small, for normal permissions across all Android versions.

Regarding dangerous permissions, we see a sudden drop in their number in Android 6.0, although this is a relatively small change compared to the increase in the number of signature permissions. This fall is because the platform developers reorganized permissions in this version to strike a balance between privacy and user burden for the newly adopted runtime permission model, where users will have to actively interact with the platform to grant dangerous permissions to apps. Afterward, however, the number of dangerous permissions is fairly stable, with only a minor increasing trend. This is reassuring, from the point of view of users and app developers, as it is desirable that the observable behavior stays consistent to avoid confusion and prevent additional adaptation burden.

Interestingly, in Android 4.2, we observe two duplicate permissions in the platform manifest, i.e., `SET_SCREEN_COMPATIBILITY` and `CHANGE_CONFIGURATION`, where the latter is introduced with different protection flags in each definition.

Changes in protection levels. As can be seen in Figure 3, the use of `signatureOrSystem` protection level

^e<https://source.android.com/docs/core/permissions/tristate-perms>

^fNote that the Android source code is made public via AOSP starting from version 1.6 and the most recent release, at the time of writing, is Android 14.

suddenly drops in Android 4.1 and this protection level (mostly) disappears afterward. What is happening behind the curtains is that the `system` flag is introduced in Android 4.1 to prepare the platform for the retirement of the `signatureOrSystem` protection level and some of the permissions with this protection level are also migrated in this version to `signature|system`, which has identical behavior.

In Android 6.0, the `signature|system` protection level is finally deprecated and the permissions with this protection level are migrated to `signature|privileged`. This change helps reduce the attack surface significantly, as it allows only the apps that reside in the designated `priv-app/` folder on the system image to be granted privileged permissions, while previously all apps residing on the system image, including unprivileged apps like Calculator, were able to obtain them.⁶

Interestingly, we observed that Android 7.0 introduces, though possibly mistakenly, two permissions with the deprecated `signatureOrSystem` protection level. These permissions are changed to `signature|privileged` only in Android 8.1.

Additionally, Android 12 introduces a new protection level called `internal`. This can be considered a “no-op” protection level, as internal permissions are granted by the platform solely based on their protection flags. This protection level was added mainly to support the role infrastructure, which allows permissions to be assigned based on the role (e.g., installer, verifier, digital wellbeing) of the requesting application.

Changes in protection level flags. Figure 4 shows the evolution of protection level and permissions flags across Android versions. It can be seen that almost all versions introduced new protection level flags. This section discusses the changes we deem the most important.

As mentioned earlier, the `system` flag was replaced with `privileged` in Android 6.0 to mitigate the attack surface posed by unprivileged system apps. In Android 5.0, `appop` was added for the management of special access permissions, which can be granted to users only with additional positive friction via the Settings. In Android 6.0, the `pre23` flag was added to provide backward compatibility for apps that target pre-6.0 Android APIs. In Android 8.1, the `instant` flag was added to support instant apps,⁸ which are apps that can be used without a full installation (e.g., to try out an app or play a game). Only certain permissions are available to instant apps.⁷

Android 12 and 13 removed the protection level flags `wellbeing`, `documenter`, and `appPredictor`

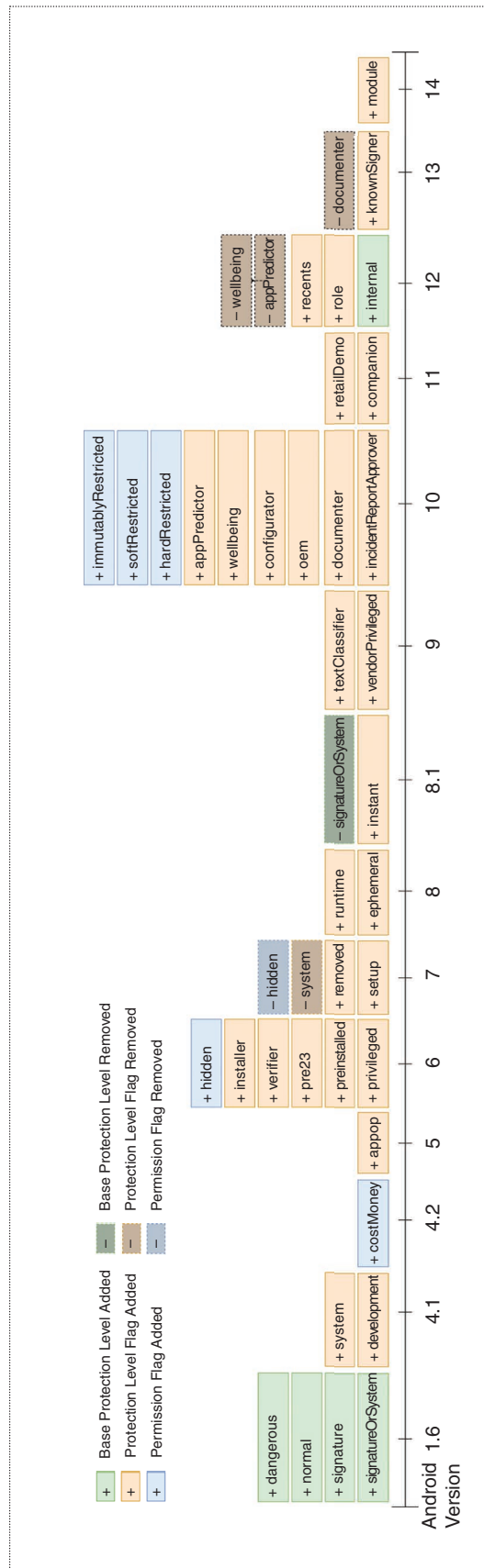


Figure 4. Evolution of protection levels, protection level flags, and permissions flags across Android versions. Note that a similar figure was produced in Almomani and Khayer⁸; here we make corrections and expand it with more recent Android versions.

⁸<https://developer.android.com/topic/google-play-instant>

because the permissions using these flags were transitioned to Android's role infrastructure, eliminating the need for these explicit flags.

Changes in permission flags. As illustrated in Figure 4, there are only a few permission flags that have been defined in Android throughout the years. Android 4.2 defined the `costsMoney` flag to indicate the operation may cost money to the user. This flag is represented in the system notifications with coin images.

In Android 6.0, the hidden flag was introduced for permissions with the normal protection level that were no longer used to protect a resource but their definitions were kept in the system for backward compatibility. Android 7.0 replaced this flag with `removed`.

Android 10 defined restricted permissions using permission flags. The platform grants `hardRestricted` permissions only to apps that fulfill special criteria defined by platform policy. In contrast, `softRestricted` permissions are granted in their full form to apps meeting the platform's special criteria; otherwise, the weaker form of the permission is granted. Finally, the `immutableRestricted` permission flag means that the restriction state of the permission can be specified only at the install time of the app and cannot change until the app is installed.¹

Interestingly, although `installerExemptIgnored` is included in the list of permission flags in the official Android documentation,² we did not find any evidence that it was declared in any of the Android versions we studied.

Changes in permission groups. Figure 5 illustrates the changes in the number of permissions groups across

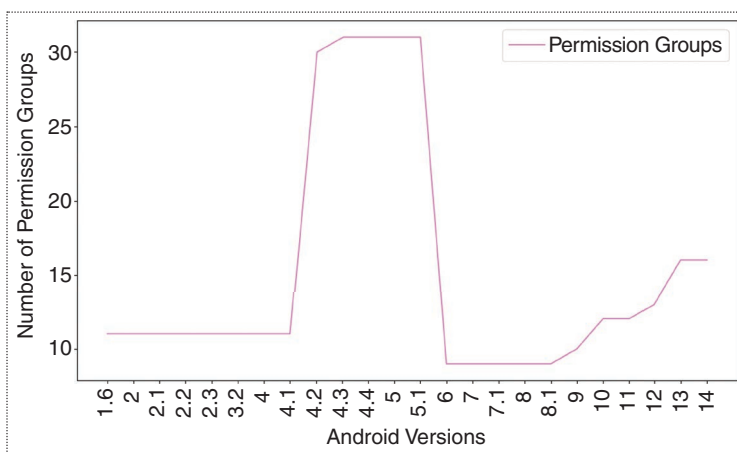


Figure 5. Number of permission groups across Android versions. Android 4.2 saw a marked rise in the number of permission groups due to organizational restructuring. This number dropped sharply in Android 6.0 with the advent of runtime permissions, designed to streamline permissions for users and mitigate warning fatigue.

Android versions. As can be seen, the number of permission groups stays stable (i.e., 11 permission groups) until Android 4.2. Afterward, we observe a sudden increase to 30, due to an effort to better organize permissions.

In Android 6.0, the number of permission groups decreased significantly from 31 to 9 due to a major restructuring of the permission system, primarily driven by the introduction of runtime permissions. More specifically, Android framework developers opted to streamline permission groups by retaining only those associated with high-risk resources that necessitate a runtime permission dialog. As such, permission groups are generally comprised of dangerous permissions; however, there are a few exceptions where a normal or a signature permission is also included in a permission group (e.g., the normal `USE_FINGERPRINT` permission in the `SENSORS` group, the privileged `ACCESS_IMS_CALL_SERVICE` permission in the `PHONE` group).

In Android 10, a new permission group named `UNDEFINED` is defined in the platform manifest. This permission group is statically assigned to permissions whose permission group will be defined later dynamically in the source code. This change comes as a part of Project Mainline,^h which allows for parts of the Android framework to be updated via Google Play Services.⁹ Since the code dealing with permissions is deemed critical, the ability to update permissions and their characteristics, including their assigned permission groups, is enabled by the Android platform.

Attacks

Previous work^{10,11,12,13,14} revealed important issues in Android's permission model. In this article, we disclose two new attacks discovered concerning Android permissions.

Unauthorized File Access Through Scoped Storage

Beginning with Android 10, apps targeting software development kit (SDK) level 29+ use scoped storage,ⁱ limiting their access solely to photos and media on the device, unlike legacy storage, which allowed access to photos, media, and all other files (e.g., documents). However, apps can opt out of scoped storage by setting the `requestLegacyExternalStorage` tag in their manifest. Based on the storage scheme used by the app, the Android platform will adjust the text in the permission request for storage accordingly, as shown in Figure 6.

We discovered a highly serious security vulnerability in the enforcement of the storage permission that was

^h<https://source.android.com/docs/core/ota/modular-system>

ⁱ<https://source.android.com/devices/storage/scoped>

introduced with the scoped storage feature. We found that an app that obtains scoped storage access can silently escalate its privileges to legacy storage and consequently obtain file access. In this case, no additional permission prompt will be shown to the user. This can be achieved in two ways:

- by downgrading the app's target SDK level to 28 (Android 9.0) or less after obtaining the storage permission for scoped storage
- by setting `requestLegacyExternalStorage` after obtaining the storage permission for scoped storage.

After completing either of these steps, the attacker would subsequently push an app update. Upon requesting the storage permission again, no permission dialog would appear, even though the user had not previously granted file access.

This issue stemmed from the Android platform's inadequate handling of app downgrades. The core problem lies in the changing semantics of the `READ_EXTERNAL_STORAGE` permission in Android 10, which no longer encompassed access to downloaded files. Consequently, an attacker who obtains the more restricted permission on Android 10 could escalate their privileges by downgrading to a version where the permission provided broader access.

This issue, which potentially affected all Android devices, was accepted as a *high-severity* security vulnerability (CVE-2021-0486) by Google and was quickly remedied due to its severity.

Side Channel to Phone Number Through Contacts

To obtain phone numbers, apps should be granted either `READ_PHONE_STATE` or `READ_PHONE_NUMBERS` for instant apps.⁷ We discovered that given the `READ_CONTACTS` and `GET_ACCOUNTS` permissions in the `CONTACTS` group, an app can exploit apps that use the device's phone number as their account name to obtain this information.

As an example, simply iterating through the accounts on the device, one can get the following:

```
Accounts: WhatsApp, com.whatsapp
Accounts: abc@example.com, com.example
Accounts: +12223334455, com.viber.voip
```

It can be observed from this output that Viber, a popular communication app with millions of downloads, uses the device's phone number as its account name. An adversary will be able to retrieve this information through the `CONTACTS` permission group, which inadvertently provides a side channel to the device's phone number. The root cause of the problem is exposing a

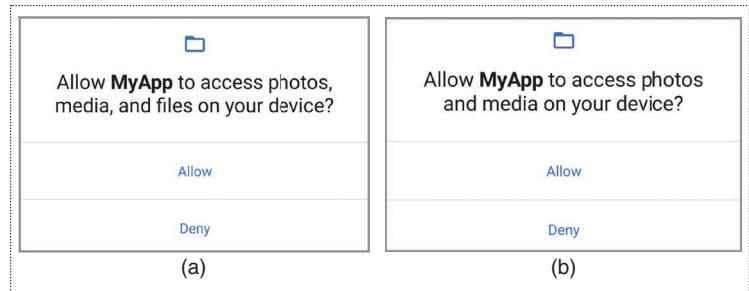


Figure 6. Permission requests for storage access on Android. On Android 9 and earlier (i.e., legacy storage), apps would be granted access to all files after they obtained the permission (a). Android 10 introduced scope storage to protect the privacy of user and app data, which also limited access to only photos and media (b).

permission-protected resource without guarding the associated data with its respective permission.

Google confirmed this issue as a *moderate-severity* vulnerability but marked it as *Won't Fix*, stating that it is the app developer's responsibility to remove any personally identifiable information from account names. We have also reported this issue to Viber, which has acknowledged it and promised to address it swiftly.

Best Practices

In this section, we offer our recommendations to platform developers when building permission features and to app developers when handling sensitive data.

Platform Developers

Preserving permission semantics. We advise caution when overriding the semantic meaning of existing permissions on Android, as this creates many edge cases revolving around app downgrades, app upgrades, and OS updates. As previously discussed in the context of the scoped storage vulnerability, if a permission's scope narrows in a future Android version, an app downgrading its target SDK could become overprivileged if this edge case is not correctly handled. Similarly, if a permission becomes less restrictive in terms of its protection level (e.g., a dangerous permission becomes a normal permission), an OS upgrade will enable users' existing apps targeting the previous SDK level to automatically obtain the permission, which can be undesired behavior in many cases.

We recommend platform developers to generally abstain from repurposing existing permissions and instead create new permissions with the intended behavior in newer Android versions. Please note that Google Play no longer allows apps to be updated with target Android versions before Android 13,⁸ further reducing this attack surface.

⁸<https://developer.android.com/google/play/requirements/target-sdk>

The evolution of storage-related permissions stands out as a commendable example of this practice.^k In Android 13, storage access is further scoped down based on the type of media access (i.e., image, video, audio). In particular, `READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, and `READ_MEDIA_AUDIO` permissions have been introduced, while still maintaining the existing `READ_EXTERNAL_STORAGE` permission and its broader access capability.

Sanitizing custom permissions. Custom permissions have become a notable avenue for attacks, enabling attackers to bypass platform safeguards and access sensitive data without user interaction.^{11,15} Here, the core issue lies in Android's lack of isolation between custom permissions (defined by app developers) and system permissions (defined by the platform), leading to potential privilege escalation. If the platform mishandles custom permissions, attackers could obtain dangerous system permissions by assigning their custom permissions to a system permission group (e.g., `PHONE`, `ACTIVITY_RECOGNITION`).

To counteract this scenario, we advise platform developers to remain vigilant for the edge cases introduced by custom permissions. In particular, it is crucial to thoroughly sanitize the state of custom permissions within a system permission group. This precautionary measure will hinder attackers from escalating their privileges by exploiting any logic flaws in the permission enforcement code.

Application Developers

Preventing unintentional exposure of sensitive data. To comply with the permission model and avoid unintentionally revealing permission-protected data, app developers should not use such data as visible identifiers in their apps. When exporting app components that handle permission-protected data, developers should ensure these components are safeguarded, with custom permissions belonging to the same permission group as the data they manage. This ensures that the Android platform will prompt the user with the relevant permission dialog.

Requesting access only to necessary data. We advise app developers to request only the permissions that are essential for their application, avoiding overwhelming users with excessive permission requests. Furthermore, we recommend utilizing the system-provided data pickers via the intent mechanism whenever possible, instead of requesting permissions. This approach enhances user

experience and respects user privacy by minimizing unnecessary data access.

This work studied the evolution of Android permissions. It showed that permissions went through significant changes throughout the years as both the Android platform and user demands evolved. It also discovered and disclosed two new attacks that involve Android permissions. Finally, it offered recommendations for platform and app developers, aiming to equip them with the knowledge necessary to proactively prevent unintentional security risks related to Android permissions. ■

Acknowledgment

We thank Jeffrey Vander Stoep, Nate Myren, René Mayrhofer, and Dave Kleidermacher for their feedback, and Dianne Hackborn for providing a comprehensive reference on Android's security and privacy design.

References

1. "Android reference: Protection level." Android Developer Guides. Accessed: Apr. 2024. [Online]. Available: <https://developer.android.com/reference/android/R.attr#protectionLevel>
2. "Android reference: Permission flags." Android Developer Guides. Accessed: Apr. 2024. [Online]. Available: <https://developer.android.com/reference/android/R.attr#permissionFlags>
3. "Android reference: Permission groups." Android Developer Guides. Accessed: Apr. 2024. [Online]. Available: https://developer.android.com/reference/android/Manifest.permission_group
4. A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner, "Android permissions: User attention, comprehension, and behavior," in *Proc. 8th IEEE Symp. Security Privacy*, 2012, pp. 1–14.
5. R. Mayrhofer et al., "The android platform security model," 2023. <https://research.google/pubs/the-android-platform-security-model-2023/>
6. Y. Zhauniarovich and O. Gadyatskaya, "Small changes, big changes: An updated view on the Android permission system," in *Proc. Res. Attacks, Intrusions, Defenses*, 2016, pp. 346–367.
7. "Create an instant-enabled app bundle." Android Developer Guides. Accessed: Apr. 2024. [Online]. Available: <https://developer.android.com/topic/google-play-instant/getting-started/instant-enabled-app-bundle#configure-permissions>
8. I. M. Almomani and A. A. Khayer, "A comprehensive analysis of the Android permissions system," *IEEE Access*, vol. 8, pp. 216,671–216,688, 2020, doi: 10.1109/ACCESS.2020.3041432.

^k<https://developer.android.com/training/data-storage/shared/media>

9. A. Acar, G. S. Tuncay, E. Luques, H. Oz, A. Aris, and S. Uluagac, "50 shades of support: A device-centric analysis of Android security updates," presented at the Netw. Distrib. Syst. Security Symp., San Diego, CA, USA, 2024.
10. A. P. Felt, H. J. Wang, A. Moshchuk, and E. Chin, "Permission re-delegation: Attacks and defenses," in *Proc. USENIX Security Symp.*, 2011, p. 22.
11. G. S. Tuncay, S. Demetriou, K. Ganju, and C. A. Gunter, "Resolving the predicament of Android custom permissions," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2018, pp. 1–15, doi: 10.14722/ndss.2018.23210.
12. D. Yeke et al., "Wear's my data? Understanding the cross-device runtime permission model in wearables," in *Proc. IEEE Symp. Security Privacy*, 2024, pp. 2404–2421, doi: 10.1109/SP54263.2024.00077.
13. G. S. Tuncay, S. Demetriou, and C. A. Gunter, "Draco: A system for uniform and fine-grained access control for web code on Android," in *Proc. ACM Comput. Commun. Security*, 2016, pp. 104–115.
14. G. S. Tuncay, J. Qian, and C. A. Gunter, "See no evil: Phishing for permissions with false transparency," in *Proc. USENIX Conf. Security Symp.*, 2020, pp. 415–432.
15. R. Li, W. Diao, Z. Li, J. Du, and S. Guo, "Android custom permissions demystified: From privilege escalation to design shortcomings," in *Proc. IEEE Symp. Security Privacy*, 2021, pp. 70–86, doi: 10.1109/SP40001.2021.00070.

Güliz Seray Tuncay is a senior research scientist in the Platforms Security and Privacy team at Google, Mountain View, CA 94043 USA. Her research interests include computer security and privacy, with a focus on mobile and usable security. Tuncay received a Ph.D. in computer science from the University of Illinois at Urbana–Champaign. She is a Member of IEEE. Contact her at gulizseray@google.com.